



MÁY HỌC (MACHINE LEARNING) - BẬC ĐẠI HỌC

Chương 6 (Phần 2): Đánh giá, Tinh chỉnh và Triển khai Mô hình

“Nếu không thể đo lường, bạn không thể tối ưu.”



Dữ liệu cân bằng (Balanced data):

Các lớp xuất hiện đồng đều.

Sử dụng: Độ chính xác (Accuracy) và ROC AUC.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

(Chú thích: TP=True Positive, TN=True Negative, FP=False Positive, FN=False Negative)



Dữ liệu mất cân bằng (Imbalanced data):

Ví dụ: 99% giao dịch thường, 1% gian lận.

Cảnh báo: Accuracy vô dụng (đoán bừa "không gian lận" cũng đúng 99%).

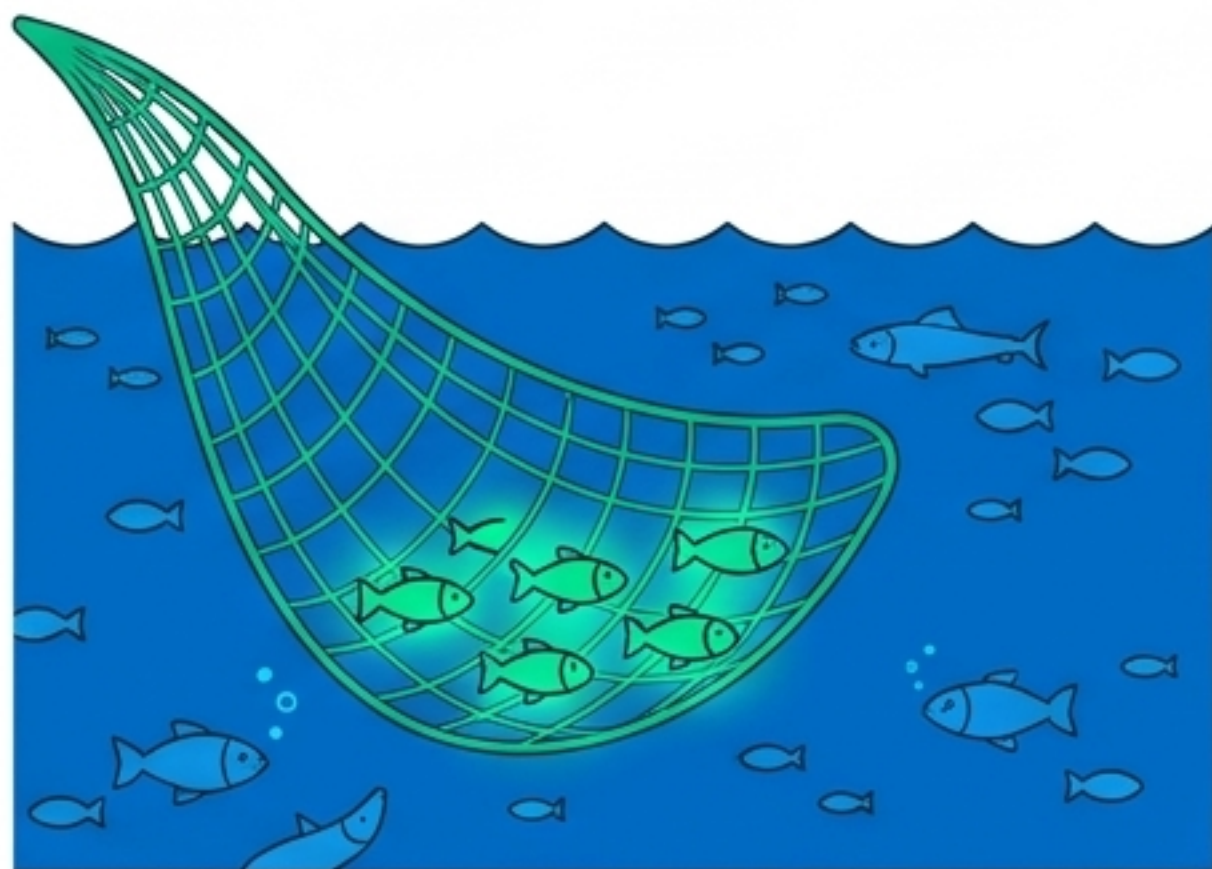
Giải pháp: Phải dùng Precision và Recall.



Độ chuẩn xác (Precision)

$$\text{Precision} = \frac{TP}{TP + FP}$$

Ý nghĩa cốt lõi: Trong số các giao dịch **MÁY CẢNH BÁO**, bao nhiêu % là gian lận thật?



Độ bao phủ (Recall)

$$\text{Recall} = \frac{TP}{(TP + FN)}$$

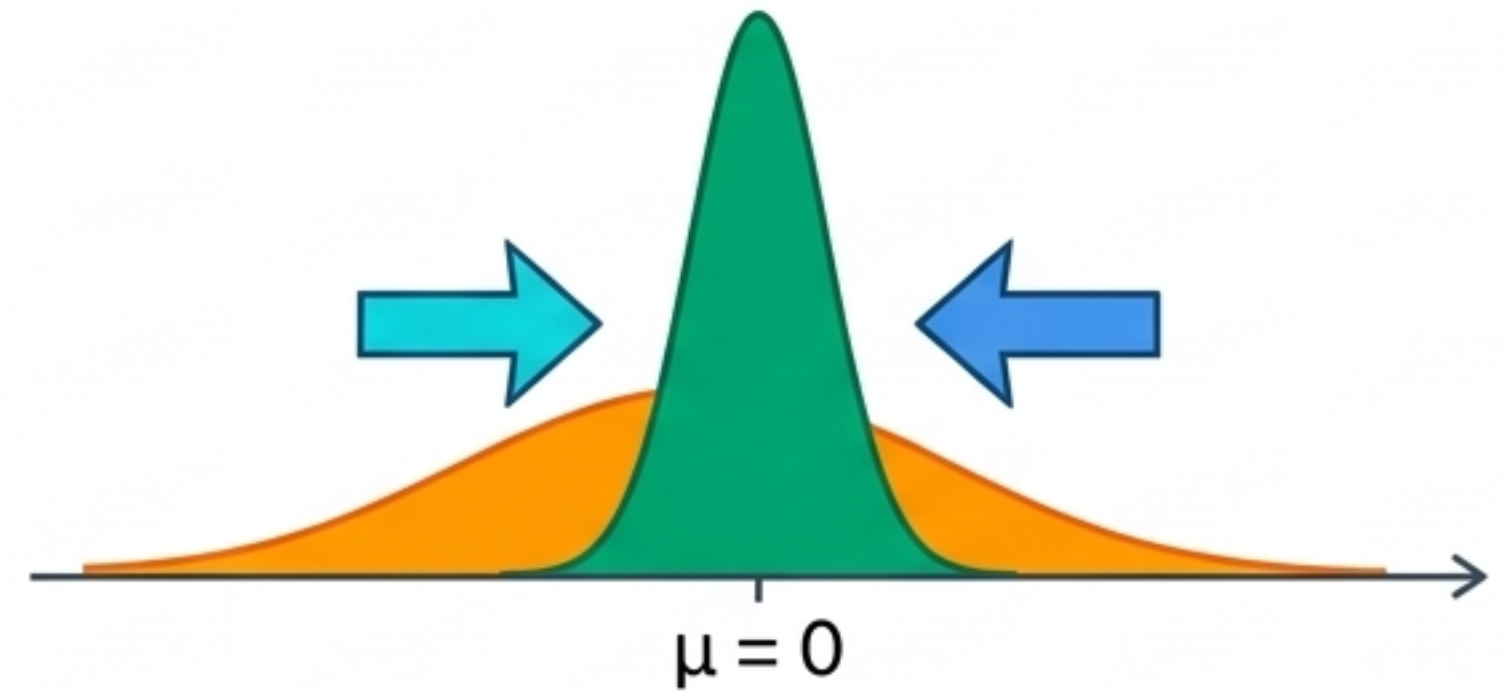
Ý nghĩa cốt lõi: Trong **TỔNG SỐ** gian lận có thật, máy tìm ra được bao nhiêu %?

Vector hóa (Vectorization)



Mạng Nơ-ron chỉ nhận Tensor chứa số thực (float32). Mọi văn bản, âm thanh, hình ảnh đều phải chuyển thành Tensor.

Chuẩn hóa (Normalization)



Tránh hiện tượng Gradient bùng nổ khi các đặc trưng có thang đo chênh lệch (vd: 0-1 và 100-200). Đưa về $\mu = 0$, $\sigma = 1$.

Listing 1: Cú pháp Chuẩn hóa cơ bản bằng NumPy

```
1 # Giả sử x là ma trận dữ liệu (samples, features)
2 # Trừ đi giá trị trung bình (Mean)
3 x -= x.mean(axis=0)
4
5 # Chia cho độ lệch chuẩn (Standard deviation)
6 x /= x.std(axis=0)
```

Xử lý Giá trị Khuyết (Handling Missing Values)

Dữ liệu phân loại (Categorical)



Tạo hần mục mới tên là **"Khuyết"** (**Missing**). Mô hình sẽ tự học cách xử lý phân lớp này.

Dữ liệu số (Numerical)

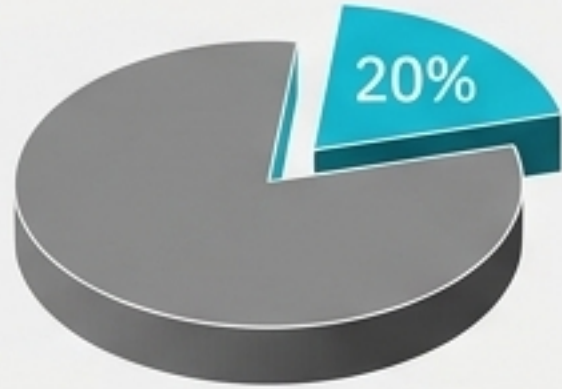


KHÔNG tự ý điền số **0** (làm gián đoạn phân bố). Thay bằng **Giá trị Trung bình** (Mean) hoặc **Trung vị** (Median).

Lưu ý cực kỳ quan trọng (Train vs Test)

Nếu dữ liệu Test có thể bị khuyết mà Train hoàn hảo → Hãy cố tình "xóa" bớt một số ô trong tập Train để mô hình tập làm quen với sự thiếu sót.

Chọn Giao thức Đánh giá: Làm sao để biết mô hình tốt mà không chạm vào tập Test?



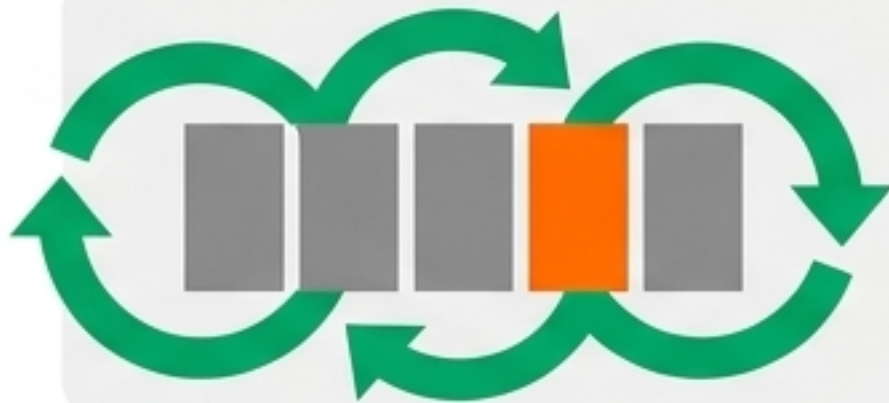
Hold-out Validation

Dành cho dữ liệu dồi dào. Cắt hẳn 20% làm tập Validation.



K-Fold Cross-Validation

Dành cho dữ liệu vừa và nhỏ. Đánh giá chéo qua K vòng lặp để lấy trung bình.



Iterated K-Fold

Lặp lại quy trình K-Fold nhiều lần với các hạt giống ngẫu nhiên (random seeds) khác nhau. Phù hợp cho dữ liệu siêu nhỏ.

Đánh bại Đường cơ sở (Beating a Baseline)

Trước khi xây Deep Learning phức tạp, hệ thống phải có **Sức mạnh Thống kê (Statistical Power)** — tức là phải tốt hơn việc "đoán mò".



Đoán mò (Guessing)

Dự đoán ngẫu nhiên
không cơ sở.

Đường cơ sở (Baseline)

Dự đoán theo nhãn đa số HOẶC
mô hình cổ điển siêu đơn giản
(Random Forest, Logistic Regression).

Deep Learning

Mạng nơ-ron phức tạp,
đòi hỏi phần cứng mạnh.

Cảnh báo chiến lược: Nếu một mô hình cực kỳ phức tạp không thắng nổi Baseline đơn giản → Giả thuyết rằng đầu vào chứa đủ thông tin để đoán đầu ra có thể đã SAI.

Bảng 6.1: Chọn Lớp Activation và Hàm mất mát (Loss Function)

Nguyên tắc: Bạn không thể tối ưu hóa trực tiếp Accuracy hay ROC AUC (vì không khả vi). Phải dùng Proxy Metric là Hàm mất mát (Loss).

Dạng Bài Toán	Lớp Activation cuối	Hàm mất mát (Loss)	Độ đo (Metrics)
● Phân loại nhị phân	Sigmoid	Binary crossentropy	Binary accuracy, ROC AUC
● Phân loại đa lớp (1 nhãn)	Softmax	Categorical crossentropy	Categorical accuracy
● Phân loại đa lớp (Nhiều nhãn)	Sigmoid	Binary crossentropy	Binary accuracy, ROC AUC
● Hồi quy vô hướng	Không có (None)	Mean squared error (MSE)	Mean absolute error (MAE)

Mở rộng Quy mô: Cố tình tạo Overfitting

Để tìm ra ranh giới tối ưu, trước tiên bạn phải biết điểm Overfitting nằm ở đâu.

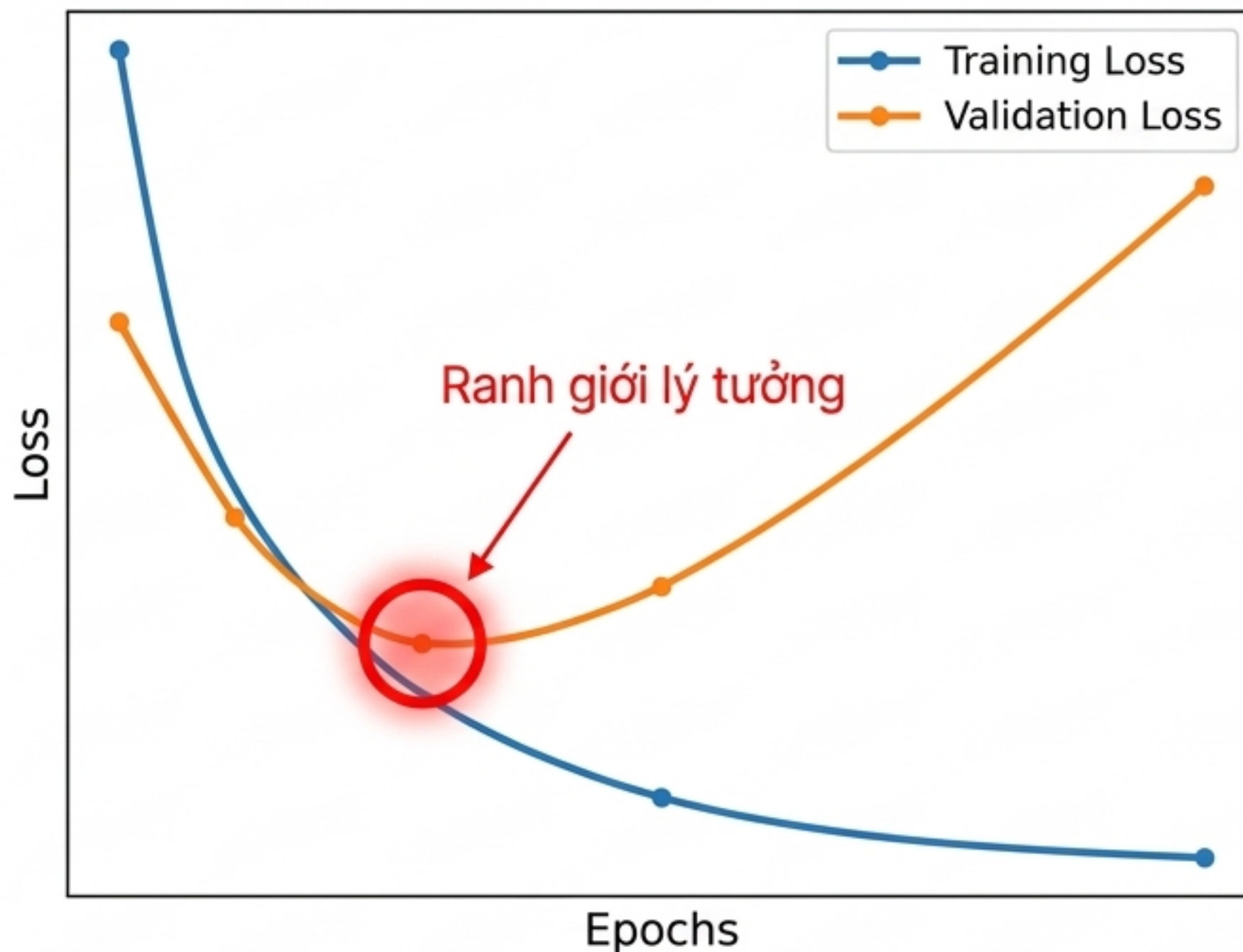
3 Bước ép mô hình Overfit:

1. Thêm các lớp (Layers) cho Mạng Nơ-ron.

2. Tăng số lượng Nơ-ron (Units) ở mỗi lớp để tăng sức chứa (Capacity).

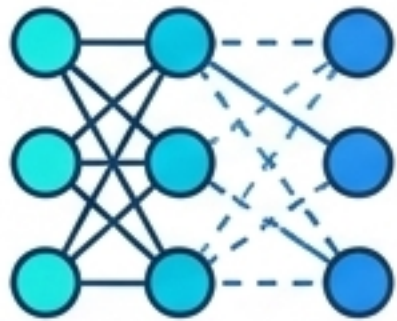
3. Huấn luyện mô hình lâu hơn (tăng số Epochs).

Quan sát: Training Loss liên tục giảm, nhưng Validation Loss sẽ vọt lên. Ranh giới lý tưởng nằm ở chính "khúc cua" này.



Tinh chỉnh và Chuẩn hóa (Regularizing & Tuning)

Đây là bước mất thời gian nhất: Gọt giũa để lấy lại hiệu suất khi đã chạm ngưỡng Overfit.



Dropout

Ngắt nơ-ron ngẫu nhiên trong quá trình huấn luyện, làm cho mô hình bớt phụ thuộc vào các đặc trưng cục bộ.



L1/L2 Regularization

Bổ sung hình phạt đối với các trọng số quá lớn trong hàm mất mát.



Siêu tham số (Hyperparameters)

Thử nghiệm tinh chỉnh Learning rate, Batch size, số Units.



CẢNH BÁO - Information Leakage: Tinh chỉnh quá nhiều lần dựa trên tập Validation sẽ dẫn đến việc mô hình **“học vẹt”** chính tập Validation này.

Quản trị Kỳ vọng với Stakeholders

Lựa chọn Ngưỡng xác suất (Threshold) thuần túy là một sự đánh đổi kinh doanh.

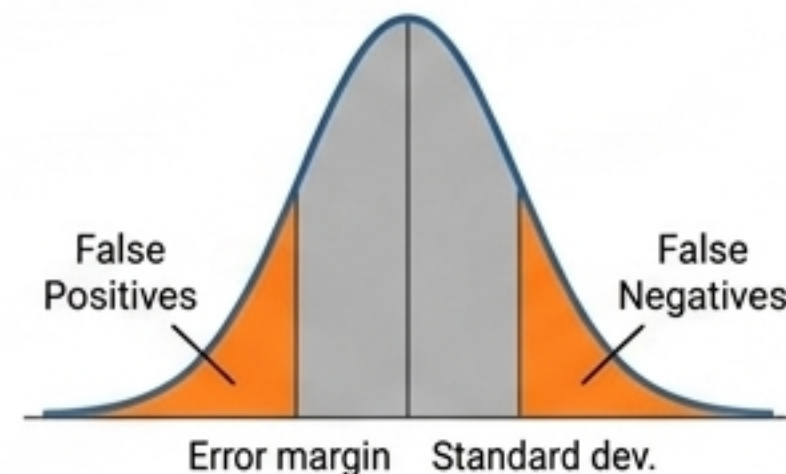


Ảo tưởng AI

Người làm kinh doanh thường hy vọng AI hoàn hảo, hiểu ngữ cảnh y như con người.



KHÔNG nói: “Mô hình chính xác 98%”



Thực tế Học máy

Mô hình chỉ khớp mẫu xác suất, LUÔN tồn tại sai số (False Positives, False Negatives).



HÃY nói: “Mỗi ngày máy sẽ bắt đúng 266 giao dịch lừa đảo, chặn nhầm 200 giao dịch thật (cần nhân viên xem lại), và bỏ sót 14 giao dịch lừa đảo”.



Xuất mô hình (Exporting an Inference Model)

Nguyên tắc: Không bao giờ chạy Python Object thô trên môi trường Production.
Cần xuất ra Đồ thị tính toán (Computation Graph).

Listing 2: Xuất mô hình ra TensorFlow SavedModel



```
# Xuất mô hình dưới dạng artifact
model.export("path/to/location", format="tf_saved_model")
# Load lại ở tiến trình khác / C++
reloaded_artifact = tf.saved_model.load("path/to/location")
predictions = reloaded_artifact.serve(input_data)
```

Listing 3: Xuất mô hình ra ONNX (Tiêu chuẩn mở)



ONNX

```
model.export("path/to/location", format="onnx")
# Load bằng ONNX Runtime
ort_session = onnxruntime.InferenceSession("path/to/location")
predictions = ort_session.run(None, input_data)
```

Các Môi trường Triển khai (Deployment Environments)



REST API (Cloud - TF Serving)

- **Cơ chế:** Client gửi HTTP, Server mạnh chạy GPU.
- ✓ **Ưu:** Sức mạnh tính toán tối đa.
- ⚠ **Nhược:** Phụ thuộc mạng, độ trễ (~500ms), rủi ro bảo mật dữ liệu cá nhân.



Trên Thiết bị (On-device - TFLite)

- **Cơ chế:** Chạy thẳng vào CPU di động (Android/iOS/Raspberry Pi).
- ✓ **Ưu:** Hoạt động offline, bảo mật cực cao, không độ trễ mạng.
- ⚠ **Nhược:** Đòi hỏi mô hình phải được nén siêu nhỏ.

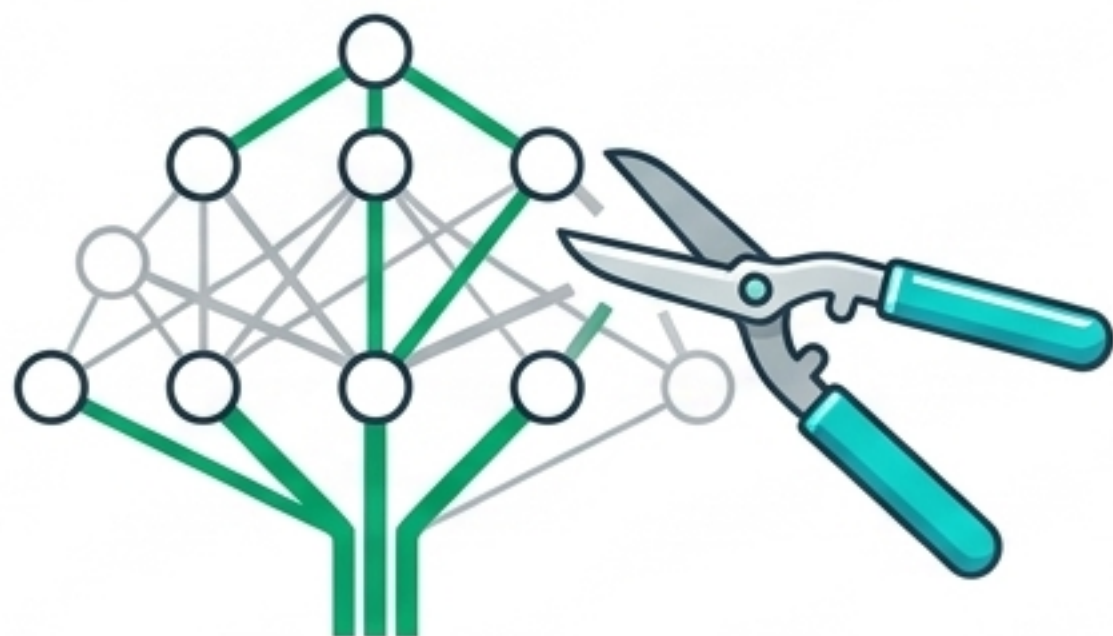


Trình duyệt (In-browser - TF.js)

- **Cơ chế:** Chạy bằng JavaScript trực tiếp trên máy khách.
- ✓ **Ưu:** Tiết kiệm chi phí server, tận dụng GPU cá nhân của người dùng.

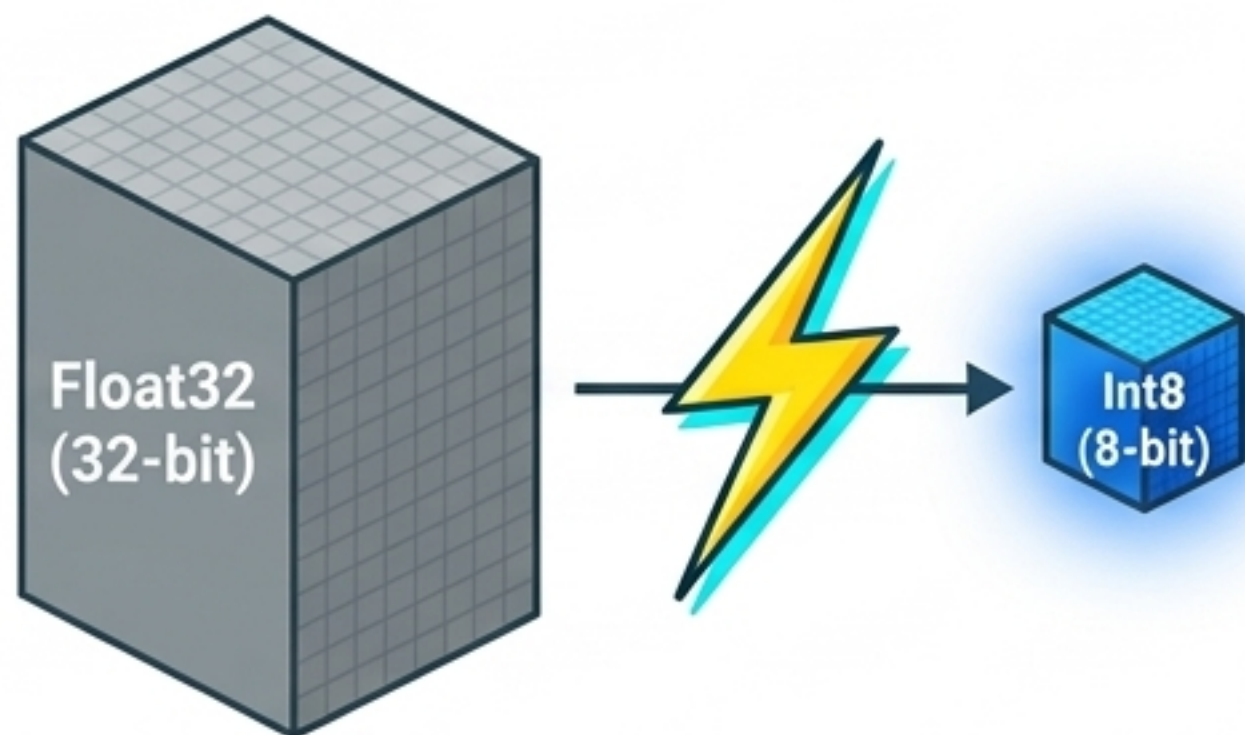
Tối ưu hóa Suy luận (Inference Optimization)

Khi đưa mô hình vào các thiết bị yếu, cần tối ưu kích thước:



Cắt tỉa trọng số (Weight Pruning)

Loại bỏ bớt các kết nối nơ-ron có trọng số quá nhỏ
→ Tiết kiệm RAM đáng kể.



Lượng tử hóa trọng số (Weight Quantization)

Mạng Nơ-ron vốn lưu số thực chính xác kép (float32).
Ta ép nó xuống số nguyên 8-bit (int8).

Kết quả: Kích thước giảm ngay 4 lần. Tốc độ tính toán tăng vọt. Độ chính xác giảm gần như không đáng kể.

```
model.quantize("int8")
```

Giám sát và Bảo trì (Monitoring & Maintaining)

Sự thật: Hiệu suất suy giảm ngay khi "Go Live" do Concept Drift (Gợi ý nhắc: vài tuần; Phát hiện lừa đảo: vài ngày).

